

数据与代码管理规范

目录

1 规范的数据与代码管理：为研发工作装上“隐形加速器”	1
2 项目文件管理规范与教程	2
2.1 项目目录结构的规范	2
2.2 文件命名的规范	3
2.3 代码管理规范	3
2.4 README.md 的撰写规范	4
2.5 项目文件整理与交接规范	5

1 规范的数据与代码管理：为研发工作装上“隐形加速器”

对于科研新手而言，如果不注意数据和代码管理规范，常常会遇到一些非常消耗精力的琐碎困扰，例如：

- 凌晨三点改完代码，第二天却找不到最新版本；
- 向老师汇报时，突然记不清某组实验参数的具体设置；
- 论文 deadline 前夜，翻遍电脑也找不到那张关键实验图的原始数据；
- 项目工作结束前整理资料，发现文件夹里堆满了“final_v2_new_final.py”……

规范的数据与代码管理，是帮助我们花时间花在刀刃上的利器，是能够为个人带来巨大长期收益的“秘密武器”。具体来说，规范的数据和代码管理的益处包括但不限于：

1、防止丢失进度，减少重复劳动，效率倍增

清晰的命名与归档让我们快速定位所需文件，告别“文件考古”，把时间花在核心创新上。同时也让我们不用担心误删代码或丢失数据。

2、培养工程素养，打造高质量的个人作品集

帮助我们掌握工业级科研协作规范，展现个人具备成熟的研发实践经验与优秀的职业素养；同时工作成果将成为个人简历中的实力凭证，为未来的深造/求职加分。

3、保障实验可复现，提高研究论文被接收的概率

目前学术会议和期刊对实验可复现的审查越来越严苛。公开共享的数据、代码以及详尽的研究方法能够有力地证明研究结果真实可靠，确保实验具有极高的复现性，能大幅提高论文被接收的概率。

4、高效地完成团队合作与交接

规范的数据与代码管理能让导师快速查看研究进度、指导实验细节；也能在需要协作完成部分任务时，让同伴快速找到所需的数据、代码和文档，避免因文件混乱导致的沟通成本增加、协作效率低下。

2 项目文件管理规范与教程

科研项目的目录结构是构建可复现研究的基础设施。混乱的目录组织会导致自己和团队成员在海量临时文件、同名脚本与不明数据源中迷失，极大增加重复梳理的时间成本。通过建立统一、模块化且逻辑清晰的目录层级，任何团队成员在接手项目文件时，都可以凭借直觉快速定位到数据处理逻辑、模型核心架构或最终的评测脚本。

2.1 项目目录结构的规范

项目文件管理的原则是“解耦”：即将“不可变的原始数据”、“动态演进的源代码”、“实验配置参数”与“自动生成的实验产出”进行严格的物理分区。例如，数据文件夹应当被视为只读的原始凭证，绝对禁止任何清洗或增强代码对原始数据集进行原地覆盖修改，所有的处理操作必须生成新的派生文件并存储于独立目录中。

以下表格详细介绍了项目目录结构的规范：

一级目录	二级目录	用途
data/	data/raw/	存储原始数据，任何人不得修改
	data/processed/	存储预处理后的中间数据，可重新生成
code/	code/src/	存放源代码，例如，机器学习算法的源代码可以这样设置三级目录： code/src/data/ code/src/models/ code/src/training/ code/src/evaluation/
	code/scripts/	存放数据预处理、训练、测试的辅助脚本
	code/notebooks/	存放探索性实验和可视化的 Jupyter Notebook 脚本
	code/results/	存放实验结果与输出，包括训练日志、控制台输出、模型文件、实验数据、可视化结果等
	code/requirements.txt	Python 依赖库与版本配置文件
docs/	docs/notes/	存放《调研报告》、《数据分析报告》、《实验方案》、《实验记录》等文档
	docs/papers/	存放论文草稿、参考文献、投稿材料等文档
	docs/slides/	存放汇报 PPT 等材料
warehouse/	自由设置即可	存放暂时闲置的代码、数据、文件
README.md	（核心！必须填写）	项目总说明文档，包含一键式的复现步骤、硬件依赖与

		环境安装说明等。 需随项目进展实时维护更新，是外部人员理解项目的唯一入口。
--	--	--

每个独立的研究小项目，必须采用以上统一的目录结构。这是团队协作的基础，确保任何人都能快速找到所需文件。

2.2 文件命名的规范

混乱的命名是导致资料丢失、无法检索的首要原因。所有文件和文件夹必须遵循通用的文件命名格式：

[内容描述]_[版本号].后缀

内容描述：简洁清晰的英文 / 拼音描述，用下划线分隔

版本号：采用 vX.Y 格式，X 为主版本，Y 为小修改。如果预计该文件后续不会发生更改，则可不使用版本号。

以下是文件命名一些其它需要注意的规范：

- 文件名中的日期：采用 YYYYMMDD 格式（如 20240520），保证排序自然。
- 内容描述：简洁清晰的英文 / 拼音描述，用下划线分隔。
- 作者缩写：姓名拼音首字母（如 zhangsan 缩写为 zs）。
- 严禁在文件名和文件夹名中包含空格、特殊标点符号（如@、#、&、()）等。

规范的文件命名示例：

```
train_data_v1.0.csv
mimic4_patient_records_v2.csv
missing_value_analysis.ipynb
train.py
run_train.sh
fig01_loss_curve.png
cvpr_crossmodal_retrieval_20260330_v3.tex
paper_draft_v2.1.pdf
```

2.3 代码管理规范

代码是实验可复现的核心，必须保证清晰、可运行、可维护。

（1）注释：核心算法、复杂逻辑必须添加清晰的注释，说明输入输出、参数含义。

（2）依赖管理：必须记录完整的 Python 环境依赖，可以在 Anaconda 使用 “pip freeze > requirements.txt” 导出 requirements.txt。

2.4 README.md 的撰写规范

README.md 是项目的“说明书”，内容需完整、清晰、简洁，面向“陌生人”撰写（假设读者完全不了解你的项目），确保无需额外讲解就能看懂项目、运行代码、获取实验结果。项目代码、实验、数据有调整时，及时修改 README.md 对应内容，避免出现“文档与实际代码不一致”的问题。

一般来说，README.md 需要包含以下内容：

项目名称	XXXX
项目简介	用 1-2 句话简要说明研究内容、核心目标，让读者快速了解项目价值，例如： 本项目针对【XXXX 研究痛点】，提出了【XXXX 核心方法/模型】，完成了【XXXX 核心实验】。
环境依赖	完整列出项目运行所需的所有环境信息，确保他人可快速搭建相同环境，例如： 一、基础环境 - Python 3.9（可根据实际修改） - PyTorch 2.0.1（可根据实际修改） 二、依赖库说明 - numpy 1.24.3 - scikit-learn 1.2.2 三、环境搭建命令
目录说明	对应项目标准目录结构，明确各核心目录功能，方便快速定位文件。
数据准备步骤	清晰说明数据相关信息，避免他人因数据问题无法复现实验。 包括原始数据信息、数据预处理步骤及具体命令、数据字段含义等内容。
运行实验步骤	提供一键可运行的命令，分训练、测试模块，明确标注参数含义（关键参数如学习率、批次大小、迭代次数等）。 包括训练命令、测试命令、参数说明等。
常见问题排查	说明实验中遇到的常见问题及解决方法，例如： 问题一：运行代码时出现 ImportError（模块缺失） 解决方法：XXXX
实验结果	用表格清晰呈现核心实验结果（如准确率、F1 值等），确保结果可追溯、可对比。
联系人	包括姓名、邮箱、备注等等。

2.5 项目文件整理与交接规范

为确保项目延续性，个人应在项目研究工作结束前一周启动交接程序，并完成以下步骤：

- （1）按照规范，整理项目数据/代码/文档
- （2）将项目文件打包，如果文件过大则上传到云盘
- （3）将项目文件传输交接给项目负责人/数据汇交员
- （4）项目负责人/数据汇交员对项目文件进行进一步地整理、备案和归档。